Cloud of Things

Module Cloud of Things – Sup'Com

FloodWatch

Alerte précoce des crues et des routes inondées

CAHIER DES CHARGES

Scope Statement

Équipe :	FloodWatch
Membres :	Guesmi baderddine – Middleware
	Ksontini Amine – Back-end & IoT
	Kamoun Ahmed – Front-end (PWA)
Remise :	24 octobre 2026
Dépôt GitHub :	`https://github.com/baderddine725/FloodWatch_cot`
Version :	1.0

École Supérieure des Communications de Tunis (Sup'Com) – Année universitaire 2026–2027

Historique des versions

Version	Date	Auteur	Modifications
0.1	20/10/2026	Équipe FloodWatch	Choix du concept et structure du document

Table des matières

1 Notre concept (justification) 2
1.1 Contexte du projet . . . 2
1.2 Problématique . . . 2
1.3 Ambitions . . . 2
1.4 Principe de fonctionnement . . . 3

2 Clients, besoins & objectifs (Scope Description) 3
2.1 Qui va adopter et utiliser la solution ? . . . 3
2.2 À quels besoins spécifiques répond le concept ? . . . 3
2.3 À quelles occasions la solution est-elle utilisée ? . . . 4
2.4 Limites fonctionnelles (hors périmètre) . . . 4
2.5 Objectifs mesurables . . . 4

3 Modèle commercial (Business Model) 4
3.1 Stratégie de commercialisation . . . 5
3.2 Politique marketing : les 4P . . . 6
3.3 Correspondance avec les 4C . . . 6

4 Livrables (Deliverables) 6

5 Contraintes (Constraints) 7
5.1 Compromis temps – coût – objectifs . . . 7
5.2 Méthodologie : eXtreme Programming (XP) . . . 8
5.3 Planning d'exécution . . . 8
5.4 Estimation des coûts : modèle COCOMO . . . 8
5.5 Contraintes technologiques . . . 11

6 Diagramme de déploiement 11

7 Hypothèses (Assumptions) 12
7.1 Mobilisation des équipes . . . 12
7.2 Hypothèses de déroulement . . . 12
7.3 Critères d'établissement du planning . . . 12
7.4 Risques pouvant engendrer des délais . . . 12

1. Notre concept (justification)

1.1 Contexte du projet

Les épisodes de pluies intenses deviennent plus courts, plus violents et plus fréquents dans le bassin méditerranéen, en particulier à l'automne. En quelques dizaines de minutes, des oueds sortent de leur lit, les réseaux d'évacuation saturent et les points bas de la voirie (passages souterrains, trémies, routes en contrebas, gués) se remplissent d'eau.

Ces zones deviennent alors de véritables pièges : un automobiliste ne peut pas estimer la profondeur de l'eau avant de s'y engager, et les véhicules peuvent caler ou être emportés. Aujourd'hui, la fermeture de ces passages repose sur l'intervention humaine (agents municipaux, protection civile), qui arrive le plus souvent *après* les premiers signalements, c'est-à-dire trop tard.

1.2 Problématique

Comment détecter la montée des eaux suffisamment tôt pour fermer automatiquement l'accès aux zones dangereuses et prévenir à la fois la population et les services d'intervention, avant que la situation ne devienne critique ?

Les difficultés actuelles sont les suivantes :

— absence de mesure en temps réel du niveau d'eau aux points critiques de la ville ;

— aucune information fiable et immédiate pour les usagers de la route ;

— réaction tardive des services, faute de vision d'ensemble des zones touchées ;

— aucune capitalisation des épisodes passés pour anticiper les prochains.

1.3 Ambitions

Développer une solution **Cloud of Things de surveillance et d'alerte précoce des crues et des routes inondées** utilisant **Jakarta EE 11 & Node-RED, MQTT, des stations connectées à base d'ESP32 et une passerelle IoT Raspberry Pi**.

La solution FloodWatch fournit :

— des **visualisations des données en temps réel** : carte interactive des stations colorées selon le niveau d'alerte, courbes de niveau d'eau et de pluie en direct ;

— des **analytiques** : historique des épisodes, comparaison entre stations, fréquence des inondations par zone ;

— des **indicateurs de performance (KPI)** : délai entre détection et fermeture, durée de fermeture des routes, taux de disponibilité des stations, nombre d'alertes par période ;

— des **alertes et notifications** pour chaque incident ou anomalie : montée des eaux, dépassement de seuil, station hors ligne, capteur défaillant ;

— une **interface de surveillance et de contrôle** : commande à distance des barrières, des feux de signalisation et des sirènes ;

— de la **prédiction** : estimation du temps restant avant débordement et du niveau de risque à court terme.

Deux applications **PWA** (*Progressive Web Apps*) sont développées avec une UI/UX optimale, installables sur smartphone et utilisables hors connexion pour la consultation :

— **PWA front-office – « FloodWatch Citoyen »** : destinée au grand public, elle affiche la carte des routes praticables ou fermées et envoie des notifications push géolocalisées ;

— **PWA back-office – « FloodWatch Pilotage »** : destinée aux opérateurs municipaux, à la protection civile et aux administrateurs, elle permet la supervision, l'acquittement des alertes, la commande des actionneurs et la configuration des stations.

1.4 Principe de fonctionnement

Chaque point critique est équipé d'une **station FloodWatch** qui mesure le niveau d'eau (capteur ultrasonique) et la pluie. Les mesures sont publiées en MQTT vers un **broker Mosquitto local** hébergé sur la **passerelle Node-RED** (Raspberry Pi), qui les filtre et les relaie, par un **pont MQTT sécurisé (TLS)**, vers le **broker MQTT** hébergé dans le Cloud. L'**API Jakarta EE 11** s'abonne aux mesures, calcule le niveau d'alerte, estime le temps avant débordement, enregistre les données et, si nécessaire, publie des commandes vers les actionneurs (barrière, feu, sirène) et des notifications vers les PWA.

Mode dégradé (sécurité locale). La fermeture de la barrière ne dépend pas de la disponibilité du Cloud : chaque station embarque une **règle de sécurité locale**. Lorsque le niveau mesuré dépasse le seuil rouge (seuils stockés en mémoire flash de l'ESP32 et mis à jour par le Cloud), la station ferme elle-même la barrière, passe le feu au rouge et déclenche la sirène, sans attendre de commande. En cas de coupure Internet ou de perte de la passerelle, la barrière se ferme donc quand même. La réouverture n'a lieu qu'après un retour du niveau sous le seuil orange pendant une temporisation configurable.

Table 1 – Niveaux d'alerte et réactions automatiques (seuils configurables par station)

Niveau	Condition (exemple)	Réaction automatique
Vert	Niveau d'eau $< 30\,\%$ du seuil critique, pas de pluie	Feu vert, barrière ouverte, mesure toutes les 30 s
Jaune	Pluie détectée ou niveau entre 30 et 60 %	Surveillance renforcée : mesure toutes les 5 s
Orange	Niveau entre 60 et 85 % ou montée rapide	Feu orange, notification aux citoyens proches et aux opérateurs
Rouge	Niveau $> 85\,\%$ ou débordement prévu en moins de 10 min	Feu rouge, fermeture de la barrière (déclenchée aussi localement par la station), sirène, alerte prioritaire à tous les opérateurs (escalade hiérarchique en option, voir Table 6)

Quel que soit le niveau, chaque station envoie un **message de vie** toutes les 15 s ; cela permet de détecter une station muette en moins d'une minute (§ 2.5).

2. Clients, besoins & objectifs (Scope Description)

2.1 Qui va adopter et utiliser la solution ?

2.2 À quels besoins spécifiques répond le concept ?

— **Sécurité des usagers** : empêcher physiquement et informer avant l'accès à une zone inondée.

— **Réactivité des services** : être alerté dès les premiers signes, sans attendre les appels.

— **Vision globale** : connaître en un coup d'œil l'état de tous les points critiques.

— **Anticipation** : estimer le temps restant avant débordement pour planifier l'intervention.

Table 2 – Clients et utilisateurs de FloodWatch

Acteur	Rôle	Application
Municipalités (client principal)	Achètent et exploitent le réseau de stations sur leur territoire	Back-office
Opérateur municipal / protection civile	Supervise les stations, acquitte les alertes, commande les barrières et sirènes	Back-office
Administrateur technique	Installe et configure les stations, règle les seuils, gère les comptes et les droits	Back-office
Citoyen, automobiliste, transporteur	Consulte l'état des routes, reçoit les alertes près de sa position	Front-office
Gestionnaires d'infrastructures (autoroutes, zones industrielles, campus)	Surveillent leurs propres points bas	Back-office

— **Traçabilité** : conserver l'historique des épisodes pour justifier les travaux d'aménagement (curage, redimensionnement des évacuations).

— **Fiabilité du réseau** : être prévenu immédiatement lorsqu'une station tombe en panne.

2.3 À quelles occasions la solution est-elle utilisée ?

— Épisodes de pluies intenses et orages (principalement de septembre à mars) ;

— Bulletins de vigilance météorologique : passage préventif en surveillance renforcée ;

— Crues soudaines d'oueds en amont des zones urbaines ;

— Surveillance continue hors épisode, pour détecter les pannes et les remontées anormales (canalisation bouchée, fuite) ;

— Analyse *a posteriori* des épisodes par les services techniques.

2.4 Limites fonctionnelles (hors périmètre)

Les éléments suivants sont **explicitement en dehors** des objectifs du projet :

— la gestion et l'organisation de l'évacuation des personnes ;

— le calcul d'itinéraires de déviation (navigation GPS complète) ;

— le pompage ou l'évacuation de l'eau ;

— la production de prévisions météorologiques : la solution *consomme* une API météo existante ;

— la modélisation hydrologique complète d'un bassin versant ;

— l'installation physique en conditions réelles : le prototype est démontré sur maquette (bacs d'eau représentant un passage souterrain et un oued) ;

— l'alimentation autonome des stations (solaire) et les réseaux longue portée (LoRaWAN), identifiés comme évolutions futures.

2.5 Objectifs mesurables

3. Modèle commercial (Business Model)

Table 3 – Objectifs du prototype

Objectif	Cible
Délai entre la mesure sur la station et l'affichage dans la PWA	< 3 secondes
Délai entre le passage en rouge et la fermeture de la barrière (commande Cloud)	< 5 secondes
Fermeture de la barrière par la règle de sécurité locale, sans liaison Cloud	< 5 secondes
Délai de notification push vers les citoyens concernés	< 10 secondes
Détection d'une station hors ligne (message de vie toutes les 15 s, alerte après 3 messages manqués)	≤ 45 secondes (< 1 minute)
Précision de l'estimation du temps avant débordement (maquette)	erreur $< 20\,\%$
Nombre de stations démontrées	2 (passage souterrain, oued)

3.1 Stratégie de commercialisation

Étude du marché

— **Cible prioritaire (B2G)** : les municipalités exposées aux inondations urbaines, en commençant par les communes du Grand Tunis et du Cap Bon, puis les autres gouvernorats.

— **Cible secondaire (B2B)** : gestionnaires d'autoroutes, de zones industrielles, de campus universitaires et de grands parkings souterrains.

— **Solutions existantes** : systèmes de télémesure hydrologique nationaux (orientés grands bassins, peu présents sur la voirie urbaine) et solutions industrielles importées de capteurs de niveau, coûteuses et sans application destinée au citoyen.

Stratégie de différenciation

— **Bout en bout** : capteur, actionneurs, Cloud et applications citoyennes dans une seule offre ;

— **Action automatique** : la station ne se contente pas de mesurer, elle ferme l'accès, même sans connexion au Cloud ;

— **Prédictif** : temps estimé avant débordement plutôt qu'une simple alarme de seuil ;

— **Faible coût** : matériel grand public, déploiement progressif station par station ;

— **Standards ouverts** : MQTT, REST et modèle W3C WoT, intégrables aux systèmes existants.

Date de lancement et jalons

Table 4 – Jalons de commercialisation

Échéance	Jalon
Décembre 2026	Prototype fonctionnel (2 stations) présenté à la fin du module

3.2 Politique marketing : les 4P

Produit (Product)

— **Station FloodWatch** : boîtier étanche avec capteur de niveau, capteur de pluie, feu de signalisation, sirène et commande de barrière ;

— **Plateforme Cloud** : API, stockage, analytique, prédiction ;

— **PWA Pilotage** (back-office) et **PWA Citoyen** (front-office) ;

— **Services** : installation, calibration, maintenance et formation des opérateurs.

Prix (Price)

Modèle **matériel + abonnement** (prix indicatifs, à affiner après le pilote) :

— kit station installé : environ 600 DT par point critique (matériel durci + installation) ;

— abonnement plateforme : environ 40 DT par station et par mois (hébergement, maintenance logicielle, mises à jour, support) ;

— application citoyenne **gratuite** pour le public, financée par les collectivités ;

— pilote gratuit de 3 mois pour la première commune partenaire.

Communication (Promotion)

— démonstrations auprès des municipalités et de la protection civile ;

— présence dans les salons et concours d'innovation (villes intelligentes, IoT, startups) ;

— campagne sur les réseaux sociaux avant la saison des pluies, vidéo de démonstration ;

— dossier de presse et partenariat avec les médias locaux lors du lancement du pilote.

Distribution (Placement)

— vente directe aux collectivités (appels d'offres et conventions de partenariat) ;

— intégrateurs et bureaux d'études en génie civil et réseaux comme partenaires installateurs ;

— PWA Citoyen distribuée par simple lien web et QR codes affichés près des points critiques, sans passage obligatoire par un magasin d'applications.

3.3 Correspondance avec les 4C

4C	Réponse de FloodWatch
Customer needs	Sécuriser les usagers et réduire le temps de réaction des services
Convenience of buying	Déploiement progressif, une station à la fois, pilote gratuit
Cost to satisfy	Coût d'une station très inférieur au coût d'un seul sinistre (véhicules, secours)
Communication	Dialogue direct avec les communes, application citoyenne comme vitrine

4. Livrables (Deliverables)

Chaque livrable est déposé sur le dépôt GitHub de l'équipe et accompagné d'une **vidéo de présentation** de moins de cinq minutes (diapositive de garde, voix commentée, sans fond sonore).

Table 5 – Liste des livrables

Livrable	Contenu	Échéance
Cahier des charges	Le présent document + vidéo de présentation	24/10/2026
Cahier conceptuel	Analyse des besoins, cas d'utilisation, diagrammes UML (communication, séquences, composants, déploiement détaillé, paquetages, classes, activités, états-transitions, temps), patrons de conception	Semaine 2 (mi-novembre)
Code source de l'API REST	Projet Jakarta EE 11 documenté, collection de tests Postman	Fin de projet (décembre)
Code source des PWA	PWA Citoyen et PWA Pilotage	Fin de projet (décembre)
Code source de la passerelle IoT	Firmware ESP32 (dont règle de sécurité locale), configuration du broker Mosquitto local et flux Node-RED exportés	Fin de projet (décembre)
Déploiement Cloud IaaS	API, broker et PWA déployés depuis GitHub (`git clone` puis `git pull`)	Fin de projet (décembre)
Applications installées	PWA installées sur le téléphone de chaque membre de l'équipe	Fin de projet (décembre)
Documentation technique	Architecture, installation, API (OpenAPI), topics MQTT	Fin de projet (décembre)
Manuel utilisateur	Guides de la PWA Citoyen et de la PWA Pilotage	Fin de projet (décembre)
Supports marketing	Plaquette commerciale, fiche produit de la station	Fin de projet (décembre)
Dossier de presse	Présentation du projet, de l'équipe et du pilote	Fin de projet (décembre)
Campagne publicitaire	Visuels pour réseaux sociaux, vidéo promotionnelle	Fin de projet (décembre)

Stratégie de diffusion du code : le code source est publié sur GitHub dans le cadre académique. Dans une optique commerciale, le firmware et les flux Node-RED pourraient rester ouverts (favoriser l'adoption et les contributions), tandis que la plateforme Cloud et le module de prédiction seraient distribués en mode service (SaaS).

5. Contraintes (Constraints)

5.1 Compromis temps – coût – objectifs

— **Temps** : contrainte la plus forte. La durée d'exécution du projet est fixée à **2 mois** (novembre et décembre 2026), soit 8 semaines de travail.

— **Coût** : budget matériel limité, uniquement des composants grand public et des offres Cloud gratuites ou étudiantes.

— **Objectifs** : ils constituent la variable d'ajustement. Le périmètre a été volontairement resserré pour tenir en 2 mois, et les fonctionnalités sont classées par priorité ; en cas de retard, les fonctionnalités « souhaitables » sont reportées sans remettre en cause la chaîne principale.

5.2 Méthodologie : eXtreme Programming (XP)

Le projet suit la méthodologie **eXtreme Programming** :

— **4 itérations courtes** de 2 semaines, chacune livrant une version fonctionnelle ;

— **user stories** rédigées avec le client (enseignant / commune fictive) et priorisées à chaque début d'itération (*planning game*) ;

— **programmation en binôme** sur les parties critiques (calcul des alertes, commandes des actionneurs, règle de sécurité locale) ;

— **tests d'abord** : tests unitaires JUnit pour l'API, tests Postman pour les points d'accès REST, tests de bout en bout sur maquette (dont coupure volontaire de la liaison Cloud) ;

— **intégration continue** : commits fréquents, chaque push laisse un code compilable ;

— **conception simple** et **refactorisation** régulière ;

— **propriété collective du code** et conventions communes (en-têtes de fichiers, nommage).

Table 6 – Priorisation des fonctionnalités

Priorité	Fonctionnalités
Indispensable	Mesure et publication MQTT, message de vie, ingestion par l'API, stockage, niveaux d'alerte, commande barrière/feu/sirène, règle de sécurité locale (fermeture autonome au seuil rouge), tableau de bord temps réel, notifications, authentification
Importante	Carte interactive, KPI essentiels (disponibilité des stations, nombre d'alertes, délai de fermeture), historique, estimation du temps avant débordement par vitesse de montée, détection de station hors ligne
Souhaitable	Prédiction avancée avec API météo, analytique comparative entre stations, escalade hiérarchique des alertes non acquittées, mode hors connexion avancé, publication sur les magasins d'applications

5.3 Planning d'exécution

Le projet dure **2 mois**, de novembre à décembre 2026, soit **8 semaines**. Le planning est exprimé en semaines (S1 correspond à la semaine du 2 novembre 2026, S8 à la semaine du 21 décembre 2026). Les échéances des livrables suivants seront ajustées selon le calendrier officiel du module.

5.4 Estimation des coûts : modèle COCOMO

L'effort est estimé avec le modèle **COCOMO intermédiaire**, en mode *organique* (petite équipe, projet bien compris) :

$$E = 3{,}2 \times (\text{KLOC})^{1{,}05} \times \text{EAF} \quad \text{(personnes-mois)} \qquad D = 2{,}5 \times E^{0{,}38} \quad \text{(mois)}$$

Taille estimée du logiciel

Seul le code écrit par l'équipe est compté (hors code généré et bibliothèques). Compte tenu de la durée de 2 mois, le périmètre a été resserré : le module de prédiction se limite à l'estimation par vitesse de montée et les PWA à l'essentiel des écrans. La règle de sécurité locale (comparaison à un seuil, quelques dizaines de lignes) est incluse dans le firmware, et la configuration du broker local ne représente pas de code applicatif.

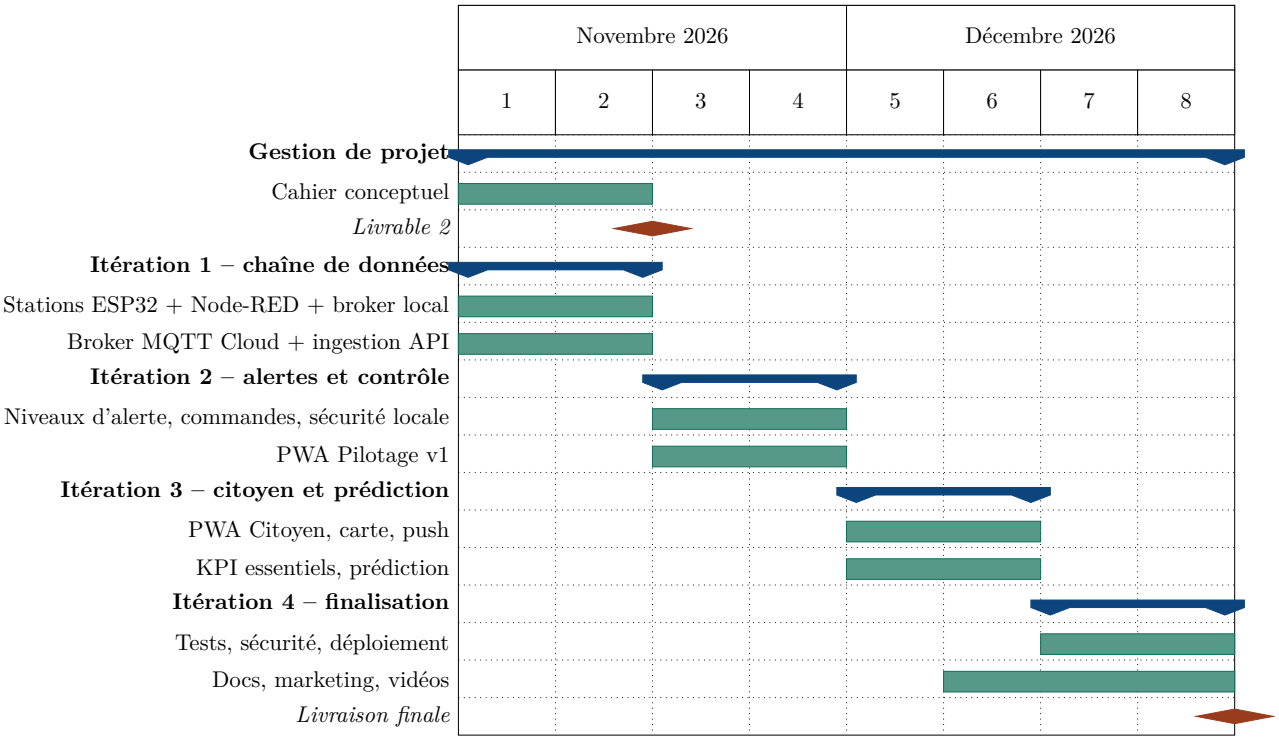


Figure 1 – Diagramme de Gantt prévisionnel (8 semaines)

Table 7 – Estimation de la taille par module

Module	**KLOC**
Firmware des stations ESP32 (C++)	0,4
Flux Node-RED de la passerelle	0,2
API REST Jakarta EE 11 (ingestion, alertes, commandes, WebSocket)	0,9
Module de prédiction	0,1
PWA Citoyen (front-office)	0,4
PWA Pilotage (back-office)	0,5
Total	**2,5**

Facteurs d'ajustement de l'effort (EAF)

La durée imposée (2 mois) étant très inférieure à la durée nominale du projet (voir plus bas), le facteur **SCED** (calendrier imposé) est ajouté aux facteurs précédents.
Remarque sur RELY (fiabilité requise) : ce facteur est laissé à Nominal car le projet livre un prototype démontré sur maquette, sans exploitation en conditions réelles (hors périmètre, § 2.4). Pour un déploiement réel, il serait relevé (Élevé, ×1,15) et l'effort augmenterait d'environ 15 % ; il sera réévalué avant la version bêta. D'ici là, la règle de sécurité locale limite les conséquences d'une panne du Cloud.

Résultats

— Effort nominal : $3{,}2 \times 2{,}5^{1{,}05} \approx 8{,}4$ personnes-mois ;

— Effort ajusté : $E \approx 8{,}4 \times 0{,}76 \approx$ **6,4 personnes-mois**, soit environ **121 hommes-jours** (1 personne-mois = 19 jours de travail) ;

Table 8 – Facteurs de coût retenus

Facteur	Niveau	Valeur	Justification
ACAP – capacité des analystes	Élevé	0,86	Formation d'ingénieur, projets antérieurs
PCAP – capacité des programmeurs	Élevé	0,86	Bonne maîtrise de Java et du web
TOOL – outils logiciels	Très élevé	0,83	IDE, GitHub, Node-RED, Postman
MODP – pratiques modernes	Très élevé	0,82	XP, tests, intégration continue
LEXP – expérience du langage	Bas	1,07	Jakarta EE 11 nouveau pour l'équipe
CPLX – complexité du produit	Élevé	1,15	Temps réel, matériel, prédiction
SCED – calendrier imposé	Très bas	1,23	Durée de 2 mois, très inférieure à la durée nominale
Autres facteurs (dont RELY)	Nominal	1,00	Voir remarque ci-dessous
EAF (produit des facteurs)		**0,76**	$0{,}62 \times 1{,}23$

— Durée nominale : $D \approx 2{,}5 \times 6{,}4^{0{,}38} \approx 5{,}1$ mois. La durée imposée de 2 mois représente donc une **compression du calendrier d'environ 60 %**, bien supérieure aux 25 % que modélise le facteur SCED « très bas » (calendrier à 75 % du nominal). Le facteur SCED ne fait que refléter le surcoût de cette contrainte ; la compression est en réalité absorbée par la réduction du périmètre, la priorisation XP (report des fonctionnalités « souhaitables ») et le travail en parallèle de toute l'équipe, soit environ $6{,}4/2 \approx 3{,}2$ personnes à temps plein.

Capacité de l'équipe : 5 étudiants à temps partiel (environ 3 jours effectifs par semaine) pendant 8 semaines, soit $5 \times 8 \times 3 = 120$ hommes-jours (environ 3 personnes à temps plein). La capacité correspond à l'effort estimé (121 hommes-jours), sans aucune marge : la priorisation XP présentée plus haut permet d'absorber tout écart en reportant si besoin des fonctionnalités « souhaitables ».

Table 9 – Table objectifs / coûts en hommes-jours

Module	Objectif couvert	Hommes-jours
Firmware ESP32	Mesure du niveau et de la pluie, message de vie, pilotage barrière, feu, sirène, règle de sécurité locale	20
Flux Node-RED	Broker local, filtrage, agrégation, pont MQTT vers le Cloud, détection de station muette	10
API REST Jakarta EE 11	Ingestion temps réel, alertes, commandes, KPI, sécurité	38
Module de prédiction	Temps estimé avant débordement (vitesse de montée), indice de risque	8
PWA Citoyen	Carte des routes, notifications push géolocalisées	20
PWA Pilotage	Supervision, contrôle, configuration, KPI essentiels	25
Total		**121**

Budget matériel du prototype

Table 10 – Coût matériel indicatif (prix à confirmer auprès des fournisseurs)

Composant	Qté	Prix unitaire (DT)	Total (DT)
Carte ESP32 DevKit	2	30	60
Capteur ultrasonique HC-SR04	2	8	16
Module capteur de pluie	2	8	16
Servomoteur SG90 (barrière)	2	10	20
LED, résistances, buzzers, câbles, plaques d'essai	1 lot	40	40
Raspberry Pi (passerelle), si non disponible à l'école	1	300	300
Maquette (bacs, carton, petit matériel)	1 lot	30	30
Total (avec Raspberry Pi)			**482**

5.5 Contraintes technologiques

- **Back-end / middleware** : Jakarta EE 11 (Java SE 17 ou supérieur), serveur d'applications compatible Jakarta EE 11 ; API REST (Jakarta RESTful Web Services), WebSocket pour le temps réel, Jakarta Persistence pour l'accès aux données, Jakarta Security pour l'authentification.
- **IoT** : protocole **MQTT** (broker Mosquitto local sur la passerelle et broker Mosquitto dans le Cloud, reliés par un pont MQTT), passerelle **Node-RED** sur Raspberry Pi, stations ESP32 en Wi-Fi. Chaque station applique localement la règle de sécurité du seuil rouge (mode dégradé).
- **Front-end** : deux **PWA** (HTML5, CSS3, JavaScript) avec manifeste, Service Worker, mode hors connexion et notifications Web Push.
- **Données** : base de données relationnelle PostgreSQL pour les stations, mesures, alertes et utilisateurs.
- **Cloud** : machine virtuelle sur un Cloud IaaS, déploiement exclusivement depuis GitHub (`git clone` puis `git pull`).
- **Sécurité** : HTTPS et WSS (TLS), MQTT sur TLS (port 8883) pour le pont entre la passerelle et le Cloud, authentification par jeton, rôles (citoyen, opérateur, administrateur). Côté local, le réseau Wi-Fi des stations est protégé (WPA2) et le broker local exige une authentification par identifiant et mot de passe ; il n'est pas exposé sur Internet.
- **Modèles imposés** : modèles IoT standards (`Thing`, `Sensor`, `Actuator`, `Payload`, `BrokerClient`), W3C WoT Thing Description et patrons de conception (Observer, Command, Stratégie, Chain of Responsibility, Decorator…).
- **Qualité** : en-tête de commentaire sur chaque fichier source (module, équipe, auteur, dates, version), commentaires de chaque fonction, nommage explicite.

6. Diagramme de déploiement

L'architecture technique globale est formalisée par le diagramme de déploiement UML ci-dessous. Il sera raffiné dans le cahier conceptuel.

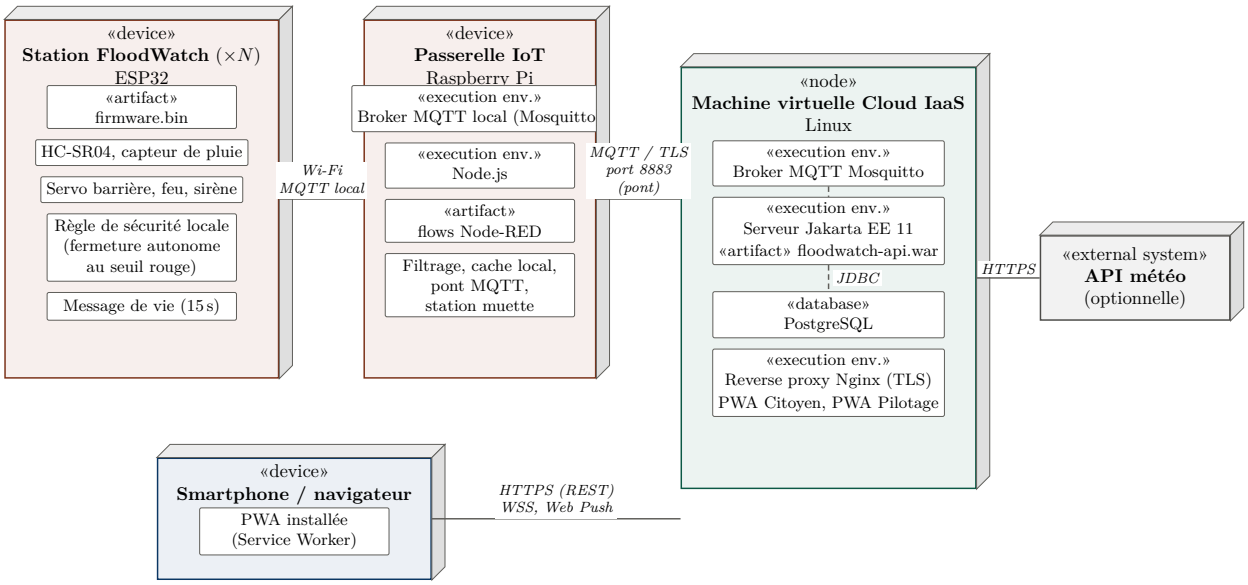


Figure 2 – Diagramme de déploiement UML de FloodWatch

7. Hypothèses (Assumptions)

7.1 Mobilisation des équipes

7.2 Hypothèses de déroulement

— Le matériel est commandé dès fin octobre et disponible au plus tard en semaine 1 (achat ou prêt par l'école).

— Une machine virtuelle Cloud IaaS (offre gratuite ou étudiante) est accessible dès la semaine 1.

— Chaque membre consacre en moyenne 3 jours effectifs par semaine au projet pendant les 8 semaines.

— Une **formation courte** (environ 1 heure) est nécessaire pour que le client ou mandataire maîtrise les rudiments du processus de bêta-tests : installation des PWA, scénario de test sur maquette, remontée des anomalies.

— L'enseignant joue le rôle du client lors des revues de fin d'itération.

— La démonstration est réalisée sur maquette ; les conditions réelles (étanchéité, alimentation, portée radio) ne sont pas testées.

7.3 Critères d'établissement du planning

— Échéance imposée du premier livrable (24 octobre 2026) et durée fixe du projet (2 mois, novembre et décembre 2026) ;

— Dépendances techniques : la chaîne de données (station, passerelle, broker, API) précède les alertes, qui précèdent les PWA ;

— 4 itérations XP de 2 semaines, chacune se terminant par une version démontrable ;

— Estimation COCOMO (121 hommes-jours) et capacité de l'équipe (120 hommes-jours) ;

— Dernières semaines (S7 et S8) réservées aux tests, au déploiement et aux vidéos ; aucune marge supplémentaire n'étant disponible, le périmètre « souhaitable » sert de variable d'ajustement.

7.4 Risques pouvant engendrer des délais

Table 11 – Description des nœuds

Nœud	Responsabilité
Station FloodWatch	Mesure du niveau d'eau et de la pluie, exécution des commandes reçues (barrière, feu, sirène), règle de sécurité locale (fermeture autonome au seuil rouge, y compris sans liaison avec la passerelle ou le Cloud), message de vie toutes les 15 s
Passerelle IoT	Broker MQTT local, collecte locale, filtrage par moyenne glissante, pont sécurisé vers le Cloud, cache local, détection des stations muettes (3 messages de vie manqués)
Machine virtuelle Cloud	Broker MQTT, API Jakarta EE 11 (règles d'alerte, prédiction, KPI, REST, WebSocket), stockage, hébergement des PWA
Smartphone / navigateur	Exécution des PWA Citoyen et Pilotage, réception des notifications
API météo (optionnelle)	Prévisions de précipitations utilisées uniquement par la prédiction avancée (fonctionnalité « souhaitable ») ; la prédiction de base repose sur la vitesse de montée du niveau

Table 12 – Périodes de mobilisation (S1 = semaine du 2 novembre 2026)

Équipe	Période	Missions principales
Toute l'équipe	S1 à S2	Cahier conceptuel, mise en place du dépôt GitHub et de la VM Cloud
Back-end / IoT (2 membres)	S1 à S6	Stations ESP32 et règle de sécurité locale, maquette, flux Node-RED, brokers MQTT (local et Cloud), module de prédiction
Middleware (1 membre)	S1 à S8	API Jakarta EE 11 : ingestion, alertes, commandes, KPI, sécurité, tests Postman
Front-end (2 membres)	S3 à S8	PWA Pilotage puis PWA Citoyen, UI/UX, notifications
Toute l'équipe	S7 à S8	Tests de bout en bout, déploiement final, documentation, marketing, vidéos

Table 13 – Analyse des risques

Risque	Prob.	Impact	Mesure de mitigation
Durée très courte (2 mois) et capacité égale à l'effort estimé	Élevée	Élevé	Périmètre resserré, priorisation XP, report des fonctionnalités « souhaitables », suivi hebdomadaire de l'avancement
Retard de livraison du matériel	Moyenne	Élevé	Commande dès fin octobre, simulateur de station en Node-RED pour avancer sans matériel
Prise en main de Jakarta EE 11	Élevée	Élevé	Prototype technique dès la semaine 1, programmation en binôme, documentation partagée

Risque	Prob.	Impact	Mesure de mitigation
Mesures ultrasoniques bruitées (vagues, condensation)	Moyenne	Moyen	Moyenne glissante dans Node-RED, rejet des valeurs aberrantes
Perte de connexion d'une station, de la passerelle ou du Cloud	Moyenne	Élevé	Message de vie toutes les 15 s et alerte « station hors ligne » après 3 messages manqués (45 s) ; règle de sécurité locale sur l'ESP32 (fermeture autonome de la barrière au seuil rouge) ; cache local sur la passerelle
Indisponibilité ou limites de l'offre Cloud	Faible	Élevé	Scripts de déploiement reproductibles depuis GitHub, VM de secours
Notifications push non supportées sur certains téléphones	Moyenne	Moyen	Tests précoces sur les téléphones de l'équipe, affichage des alertes dans l'application en repli
Données insuffisantes pour la prédiction	Élevée	Faible	Commencer par l'estimation par vitesse de montée, générer des épisodes simulés
Indisponibilité d'un membre (examens de fin de semestre, maladie)	Moyenne	Moyen	Propriété collective du code, binômes, priorisation XP

FloodWatch – protéger les usagers avant que l'eau ne monte.